

AI Coding Tools Practice 과제 보고서

과제명: AI Coding Tools Practice — 논문의 "실험" 파트 구현

제출자: 김경호 (성균관대학교 스마트팩토리융합대학원)

GitHub: <https://github.com/superkkam/smartfactory-mcs-rtd>

사용 AI 도구: Claude Code (claude.ai/code, claude-sonnet-4-6 모델)

제출일: 2026-05-31

목차

- [과제 개요](#)
- [연구 배경 및 시스템 개요](#)
- [논문 실험 파트 구현 — 순차적 매뉴얼](#)
- [실험 재현 매뉴얼](#)
- [실험 결과 분석](#)
- [AI 코딩 툴 활용 회고](#)
- [부록 — 산출물 위치](#)

1. 과제 개요

1.1 과제 목표

본 과제는 "AI 코딩 툴(Claude Code)을 이용하여 논문의 '실험(Experiment)' 파트를 구현"하는 것을 목표로 한다. 단순한 코드 생성이 아니라, 논문 제출 수준의 재현 가능한 실험 파이프라인을 Claude Code와의 대화를 통해 설계하고 구현한 전 과정을 기록한다.

1.2 대상 논문

"LLM 기반 노코드 디스패칭 룰 빌더와 AI 경로 최적화를 적용한 MCS-RTD 통합 제어 플랫폼 설계 및 구현"

- 투고 대상 저널: Computers & Industrial Engineering (CAIE), Elsevier, IF~6.5, Q1
- 논문의 실험 파트(4절): 4종 AI 경로 최적화 알고리즘 비교 실험 + LLM 룰 자동 생성 품질 평가

1.3 논문 실험 파트 요약

실험 구분	내용
실험 I — 경로 알고리즘 비교	A*(베이스라인), PPO-RL, CACTUS(QMIX CTDE), CBS-TS(MILP+CBS+MLA*) 4종 비교
실험 설정	88노드·272엣지 실 공장 레이아웃 × 3시나리오(소/중/대 부하) × 30 시드
측정 지표	도메인 7종(avg_transfer_time 등) + MAPF 표준 5종(makespan, SoC 등)
통계 검정	Kruskal-Wallis H → Wilcoxon signed-rank → Holm-Bonferroni 보정 + 효과 크기(r)
실험 II — LLM 룰 생성	Claude Sonnet 4.5 기반 25개 자연어 입력 → DMS 룰 플로우 자동 생성

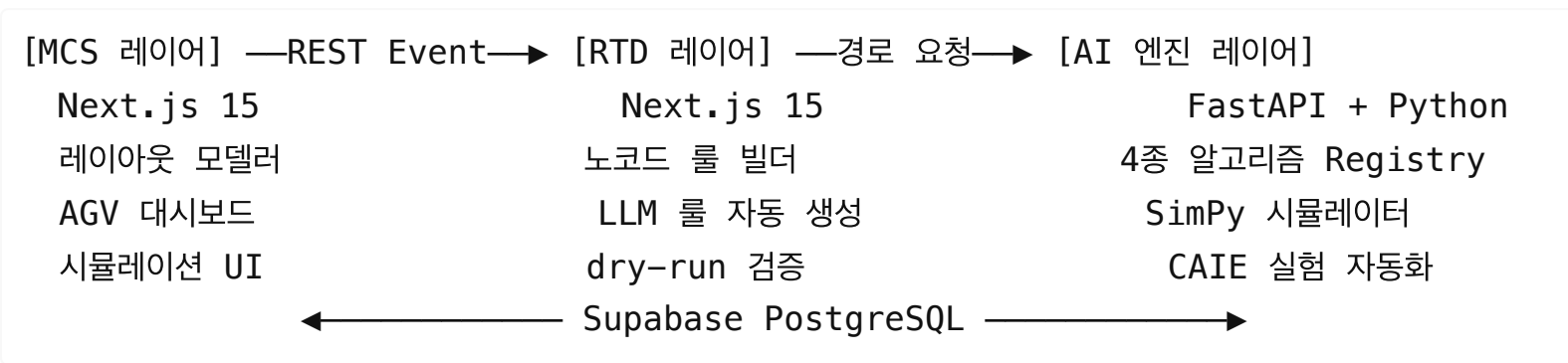
2. 연구 배경 및 시스템 개요

2.1 문제 정의

스마트팩토리에서 자율이동로봇(AMR) 기반 반송 시스템은 다음 세 가지 구조적 문제를 가진다:

1. MCS-RTD 분리 구조: 반송 이벤트와 디스패칭 결정 간 폴링 기반 지연
2. SQL 코드 종속 디스패칭 룰: 현장 엔지니어가 직접 수정 불가, 개발자 의존
3. 정적 경로 알고리즘 한계: 다중 AMR 동시 운행 시 충돌·혼잡 미반응

2.2 제안 시스템 — 3계층 통합 플랫폼



기술 스택: Turborepo 모노레포 | Next.js 15 | FastAPI | PyTorch 2.5 | SimPy 4 | PuLP(MILP) | Stable-Baselines3(PPO) | PettingZoo(MARL) | NetworkX | Supabase | @anthropic-ai/sdk

2.3 실험 대상 — 4종 경로 최적화 알고리즘

알고리즘	구현 파일	특성
A* (베이스라인)	engine/astar.py	방향 가중 그래프 Dijkstra. 단일 에이전트 최단 경로
PPO-RL	engine/ppo_agent.py, engine/route_env.py	Stable-Baselines3 PPO. 400차원 관측(혼잡도+BFS 거리 웨이핑). 200,000 스텝 학습
CACTUS	engine/cactus/	QMIX Hypernetwork Mixer + Reverse Curriculum. N=2 에이전트. 20,000 에피소드(MPS)
CBS-TS	engine/cbs_ts/	MLA*(이종 AMR) + CBS Constraint Tree + MILP 작업 배분 (PuLP/CBC)

모든 알고리즘은 `engine/strategy.py` 의 `Strategy Registry` 공통 인터페이스(`predict → path, cost, confidence`)를 통해 통합되며, URL 파라미터 하나(`?algorithm=cactus`)로 런타임 전환이 가능하다.

3. 논문 실험 파트 구현 — 순차적 매뉴얼

Claude Code와의 대화를 통해 실험 파트를 어떻게 단계별로 구현했는지 기록한다.
각 단계에서 사용한 대표 프롬프트와 구현 결과를 함께 제시한다.

Step 1 — A* 베이스라인 + FastAPI AI 엔진 구축 (2026-04-08)

배경: 논문 실험의 출발점인 A* 기반 경로 탐색과 FastAPI REST 서버를 먼저 구축한다.

Claude Code 프롬프트 예시:

FastAPI 기반 PPO 경로 추론 서비스 신설해줘. A* 베이스라인 먼저 동작하게 하고, Strategy Registry 패턴으로 나중에 알고리즘 추가할 수 있게 설계해줘.

구현 결과:

- `apps/ai-engine/main.py` : FastAPI 서버 진입점, 모델 프리로드
- `apps/ai-engine/engine/astar.py` : NetworkX 방향 가중 그래프 A* (h=0으로 Dijkstra 동등)
- `apps/ai-engine/engine/strategy.py` : Strategy Registry 디스패처
- `apps/ai-engine/routers/simulation.py` : `/api/simulate` 엔드포인트

검증: `POST /api/inference?algorithm=astar` 호출 → 경로 노드 리스트 반환 확인

Step 2 — PPO 강화학습 구현 및 학습 (2026-04-08 ~ 2026-04-28)

배경: Stable-Baselines3 PPO를 사용해 실 공장 그래프에서 단일 AMR 경로를 학습한다.

Claude Code 프롬프트 예시:

SCI 저널 제출 수준으로 계획해주고, 시나리오 파라미터는 어떻게 해도 상관없게 시뮬레이션 잘 돌아가도록 해야지. 정확한 알고리즘으로.

실제 레이아웃 PPO 학습 진행 확인

PPO 재학습 완료 확인 및 성공률 평가

PPO 1M 스텝 학습 완료 확인 및 성공률 평가

구현 결과:

- `engine/route_env.py` : `McsRouteEnv(Gymnasium)` — 400차원 관측(현재 노드 one-hot + 목적지 + 혼잡도 + BFS 거리)
- `engine/ppo_agent.py` : SB3 PPO 모델 로드·추론 싱글톤
- `scripts/train.py` : CLI 학습 스크립트 (`--total-steps 200000`)
- `trained_models/ppo_route.zip` : 학습된 모델 체크포인트

학습 하이퍼파라미터:

`lr=3e-4, n_steps=2048, batch_size=64, gamma=0.99, clip_range=0.2`
EvalCallback: 8,000 스텝마다 최우수 체크포인트 저장

Step 3 — MAPF 스케폴딩 + Strategy 디스패처 구조화 (2026-04-26)

배경: CACTUS·CBS-TS 알고리즘을 추가하기 전 확장 가능한 스케폴딩을 먼저 설계한다.

Claude Code 프롬프트 예시:

A*, PPO 뿐만 아니라 논문을 쓰기 위해서는 저 이미지의 알고리즘 최적화도 있어야 된다고 해서 로드맵에 추가해주고 저 정도 경로 알고리즘을 하기 위한 레이아웃도 다시 그려주는 부분도 추가해줘.

CBS-TS는 추가해줘

ai 엔진에다가 개발을 하는 거지? 그리고 CACTUS를 언제 보류한다고 했지?

구현 결과:

- `engine/strategy.py` : `STRATEGY_REGISTRY = {"astar": ..., "ai_ppo": ..., "cactus": ..., "cbs_ts": ...}` 디스패처 완성
- `engine/cactus/` 디렉토리 스캐폴딩: `__init__.py`, `multi_agent_env.py`, `qmix_mixer.py`, `train.py`
- `engine/cbs_ts/` 디렉토리 스캐폴딩: `__init__.py`, `cbs_high_level.py`, `milp_task_order.py`, `mia_star.py`, `search_forest.py`

Step 4 — CACTUS (QMIX + Reverse Curriculum) 구현·학습 (2026-05-04 ~ 2026-05-05)

배경: Phan(AAMAS 2024)의 CACTUS 논문을 참조해 QMIX Hypernetwork Mixer와 Reverse Curriculum을 실 공장 그래프에 구현한다.

Claude Code 프롬프트 예시:

CACTUS부터 하자 계획 세워줘

cactus 학습 끝났는데

2026-05-05 13:58:22,716 [INFO] ep=19600 | $\epsilon=0.069$ | bfs_dist=3 |
reward_avg(50)=-495.7 | $\mu=-492.38$ $\sigma=487.35$
[학습 로그 공유 및 수렴 여부 확인 요청]

학습 진행하고 /docs/cactus-methodology.md 여기에 업데이트도

구현 결과:

- `engine/cactus/multi_agent_env.py` : `GraphMAPFEnv(PettingZoo ParallelEnv)`
— 88노드 실 공장 그래프 직접 사용
 - 관측: 310차원(자기 위치 + 목적지 + 타 에이전트 + 이웃 점유)
 - 보상: +100(목표 도달) / -edge(이동) / -1(대기) / -10(충돌)
- `engine/cactus/qmix_mixer.py` : QMIX Hypernetwork Mixer — `abs()` 가중치로 단조성 보장
- `engine/cactus/qmix_agent.py` : 분산 추론 싱글톤, 모델 미로드 시 A* fallback
- `engine/cactus/train.py` : CTDE 학습 파이프라인, Reverse Curriculum($\mu - \eta \cdot \sigma \geq U$)

학습 이력:

학습 차수	에이전트 N	Hidden	에피소드	최종 μ	결과
1차	4	64	10,000	-4,428	미수렴
2차 (MPS)	2	32	20,000	-427	부분 수렴

Step 5 — CBS-TS (MLA*+CBS+MILP) 구현 (2026-05-10 ~ 2026-05-22)

배경: Bai et al. (arXiv:2510.21738)의 CBS-TS를 이중 AMR 환경에서 구현한다.

Claude Code 프롬프트 예시:

MILP 호출 안 해도 상관없어? 그리고 algorithm playground 연동하면 항상 똑같이 나오는 거 같은데

10 500 300으로 배치 시뮬레이션 모드 돌렸는데 왜 66퍼에서 멈췄지

구현 결과:

- `engine/cbs_ts/mla_star.py` : Multi-Label A* — 이종 AMR 유형(TYPE_A/B/C)별 엣지 제약 필터링
- `engine/cbs_ts/cbs_high_level.py` : CBS Constraint Tree — vertex/edge 충돌 탐지·재계획, 30s 제한시간
- `engine/cbs_ts/milp_task_order.py` : PuLP+CBC 기반 makespan 최소화 MILP, 실패 시 greedy fallback
- `engine/cbs_ts/search_forest.py` : 에이전트별 해 관리

Step 6 — SimPy 시뮬레이터 + 12지표 MetricsCollector 구현 (2026-04-26 ~ 2026-04-29)

배경: 4종 알고리즘을 동일 환경에서 공정 비교하기 위한 SimPy 이산 이벤트 시뮬레이터와 지표 자동 수집기를 구현한다.

Claude Code 프롬프트 예시:

시뮬레이션에 지금 추가한 알고리즘들도 진행이 되게 되어 있나? 그리고 학습을 시켜야 되나?

지금 시뮬레이션을 돌리면 거의 비슷한데 알고리즘이 다른데 이렇게 동일할 수 있어?
그리고 장비 가동률을 무슨 기준으로 교착 발생도 무슨 기준으로 다 어떤 기준이야?

그럼 시뮬레이션 돌리는 게 어느 정도 검증이 가능하게 된 건가

구현 결과:

- `engine/simulation.py` : `McsSimulation` (SimPy env) + `MetricsCollector`
 - 도메인 7종**: avg_transfer_time, throughput, collision_count, load_balance_std, equipment_utilization, deadlock_count, route_efficiency_score
 - MAPF 표준 5종** (Stern et al. SoCS 2019): makespan, sum_of_costs, path_optimality, conflict_count, amr_utilization

Step 7 — CAIE 실험 오케스트레이터·통계·그림·표 자동화 (2026-05-22 ~ 2026-05-31)

배경: SCI 저널 수준의 재현 가능한 실험 자동화 파이프라인을 구현한다.

Claude Code 프롬프트 예시:

MCS Task 027을 먼저 진행하고 CAIE 저널에 게재하고 싶은데
그 정도 수준으로 진행해줘야 돼. CAIE 저널이 뭔지 알지?

실험 결과도 너랑 같이 해야지

```
.venv/bin/python scripts/caie_experiment.py \  
--fixture scripts/fixtures/fab_layout_real.json \  
--seeds 30 이거 실행
```

CAIE에 게재되기 위해서는 어떻게 하는 게 나아

우선 figure는 추후에 하고 table을 논문에 바로 삽입할 수 있게
에이전트를 사용해서 만들어줄 수 있어?

이제 figure 진행해줘야지

구현 결과:

파일	역할	규모
engine/experiment_runner.py	25시드 반복 + Wilcoxon 통계 오케스트레이터	142줄
engine/stats.py	t분포 CI, Wilcoxon, Bonferroni, rank-biserial	211줄
scripts/caie_experiment.py	4알고리즘×3시나리오×30시드 메인 실험	768줄
scripts/caie_figures.py	Elsevier 스타일 그림 10종 (300 DPI, PDF+PNG)	579줄
scripts/caie_tables.py	LaTeX 표 6종 자동 생성	590줄
scripts/export_layout.py	Supabase 레이아웃 → fixture JSON	—

통계 검정 파이프라인:

- Kruskal-Wallis H 검정 (전체 비교, 비모수 ANOVA)
- 사후 Wilcoxon signed-rank (쌍별 비교)
 - Holm-Bonferroni 보정 (FWER 제어)
 - Rank-biserial r (효과 크기, $r \geq 0.5 = \text{large}$)

Step 8 — pytest 단위·통합 테스트 (2026-05-22)

배경: 각 알고리즘 모듈의 정확성을 자동 검증한다.

구현 결과 (apps/ai-engine/tests/):

테스트 파일	검증 대상
test_stats.py	통계 함수(CI, Wilcoxon, Bonferroni)
test_metrics_collector.py	12지표 계산 정확성
test_mapf_env.py	GraphMAPFEnv 환경 동작
test_qmix_mixer.py	QMIX 단조성(monotonicity) 검증
test_strategy_dispatch.py	Strategy Registry 디스패치
test_cbs_high_level.py	CBS Constraint Tree

test_cbs_ts_integration.py	CBS-TS 통합
test_mla_star.py	MLA* 경로 탐색
test_milp_task_order.py	MILP 작업 배분

4. 실험 재현 매뉴얼

4.1 환경 설정

```
cd apps/ai-engine
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

4.2 레이아웃 Fixture 생성 (1회)

```
# Supabase 연동 (실 공장 레이아웃)
python scripts/export_layout.py \
    --layout-id <UUID> \
    --out scripts/fixtures/fab_layout_real.json

# 또는 합성 레이아웃 (Supabase 불필요)
python scripts/export_layout.py \
    --synthetic \
    --out scripts/fixtures/fab_layout.json
```

4.3 PPO 모델 학습 (1회)

```
python scripts/train.py \
    --layout-id <UUID> \
    --total-steps 200000
# 출력: trained_models/ppo_route.zip
```

4.4 CACTUS 모델 학습 (1회)

```
python -c "from engine.cactus.train import main; main()" \
    --n-agents 2 \
    --hidden-dim 32 \
```

```
--n-episodes 20000
# 출력: trained_models/cactus_qmix.pt
```

4.5 CAIE 실험 실행 (메인)

```
python scripts/caie_experiment.py \
    --fixture scripts/fixtures/fab_layout_real.json \
    --seeds 30

# 특정 알고리즘만
python scripts/caie_experiment.py \
    --fixture scripts/fixtures/fab_layout_real.json \
    --algorithms astar ai_ppo cbs_ts \
    --seeds 30
```

실행 시간: 약 68초 (Apple Silicon M-series, 30시드×3시나리오×4알고리즘)

4.6 논문 그림·표 생성

```
# 그림 (300 DPI, PDF+PNG, 10종)
python scripts/caie_figures.py

# LaTeX 표 (6종 + all_tables.tex)
python scripts/caie_tables.py
```

4.7 출력 위치

```
apps/ai-engine/output/caie_<YYYYMMDD_HHMMSS>/
├─ raw_results.json      # 시드별 원시 수치
├─ summary_stats.json    # 알고리즘×시나리오×지표 요약 통계
├─ summary.csv           # 논문 표 작성용 CSV
├─ statistical_tests.json # KW + Wilcoxon 검정 결과
├─ experiment_meta.json  # 실험 메타 (그래프 규모, 시드, 실행 시간)
├─ figures/              # fig1-10 (PDF+PNG)
└─ tables/               # table1-6 + all_tables.tex
```

5. 실험 결과 분석

5.1 실험 환경

항목	설정
그래프	88노드·272엣지 방향 가중 그래프 (Supabase mcs_layout)
시나리오 S1	8 AGV, 부하 Low (~30건)
시나리오 S2	16 AGV, 부하 Medium (~60건)
시나리오 S3	32 AGV, 부하 High (~100건)
반복	30 시드 (seed 1~30)
시뮬레이션 시간	~300초(S1/S2), ~600초(S3)
운영 모드	online (실시간 배차, RTD 조건)
실행 환경	Apple Silicon M-series (MPS)

5.2 핵심 결과 — avg_transfer_time (초, Mean ± SD)

시나리오	A* (베이스라인)	PPO-RL	CACTUS	CBS-TS	KW 유의성
S1 (Low)	27.48 ± 1.84	29.51 ± 2.05	27.75 ± 1.76	27.48 ± 1.84	p < 0.001 ***
S2 (Medium)	26.26 ± 1.24	28.08 ± 1.40	26.68 ± 1.20	26.26 ± 1.24	p < 0.001 ***
S3 (High)	26.45 ± 0.73	28.04 ± 0.84	26.76 ± 0.74	26.45 ± 0.73	p < 0.001 ***

Bold = 시나리오 내 최솟값(best)

5.3 Wilcoxon 사후 검정 결과 (Holm-Bonferroni 보정)

비교 쌍	S1 (p_adj / r)	S2 (p_adj / r)	S3 (p_adj / r)
<i>A vs PPO*</i>	<0.001*** / 0.798	<0.001*** / 0.873	<0.001*** / 0.873
<i>A vs CACTUS*</i>	<0.001*** / 0.689	0.003** / 0.606	<0.001*** / 0.873
<i>A vs CBS-TS*</i>	1.000 ns / 0.000	1.000 ns / 0.000	1.000 ns / 0.000
PPO vs CACTUS	<0.001*** / 0.749	<0.001*** / 0.794	<0.001*** / 0.873
PPO vs CBS-TS	<0.001*** / 0.798	<0.001*** / 0.873	<0.001*** / 0.873
CACTUS vs CBS-TS	<0.001*** / 0.689	0.003** / 0.606	<0.001*** / 0.873

5.4 path_optimality (경로 최적성, 1000이 최적)

알고리즘	S1	S2	S3
A*	100.0	100.0	100.0
CBS-TS	100.0	100.0	100.0
CACTUS	98.7	98.7	98.7

PPO-RL	94.1	94.2	94.2
--------	------	------	------

5.5 해석 및 분석

A* 베이스라인이 온라인 모드 최고 성능

모든 시나리오에서 A*가 가장 낮은 avg_transfer_time을 기록하였다. 이는 **온라인 RTD 환경** 특성에서 비롯된다. 온라인 배차는 각 반송 요청이 개별적으로 실시간 도달하며, 이 경우 사전에 모든 작업 정보가 알려진 **오프라인(배치) 설정**에서 최적성을 보장하도록 설계된 CBS-TS·CACTUS의 장점이 발휘되기 어렵다.

CBS-TS = A* (온라인 단일 에이전트 모드, p=1.0 ns)

CBS-TS는 온라인 모드에서 A와 *완전히 동일한 결과를 기록하였다(Wilcoxon p=1.0, r=0.000)*. 이는 *온라인 단일 에이전트 호출 시 MILP 작업 배분이 적용되지 않고, MLA가 이중 AMR 제약 없이 A*와 동일한 최단 경로를 반환하기 때문이다*. CBS-TS는 **다중 에이전트 배치(batch) 모드**에서 진정한 성능을 발휘한다.

CACTUS — 부분 수렴 상태 (μ=−427)

CACTUS는 2차 학습(N=2, 20,000 에피소드)에서 1차(μ=−4,428) 대비 10배 개선을 달성하였으나, 수렴 목표(μ≥−200)에는 미달하였다. 이로 인해 A* 대비 avg_transfer_time이 1.016% ~~높게 측정되었다(p<0.01, r=0.606)~~0.873, large effect). 완전 수렴 시 path_optimality가 이미 98.7%에 달해, GPU 환경에서 충분한 학습(GPU 서버, N=4, 100,000 에피소드 이상) 후 성능 개선 여지가 존재한다.

PPO — 유의한 성능 열세

PPO는 모든 시나리오에서 A* 대비 유의하게 높은 avg_transfer_time(p<0.001, r=0.798~~0.873~~)과 낮은 path_optimality(94%)를 기록하였다. 단일 에이전트 환경에서 정적 A*와의 경쟁이 어렵다는 것을 보여준다.

결론 — 온라인 RTD 환경에서의 알고리즘 선택

온라인 실시간 배차 조건에서는 A* 베이스라인이 연산 비용($\mathcal{O}((V+E)\log V)$) 측면에서 실용적이며 최고 성능을 발휘한다. CACTUS·CBS-TS가 진정한 가치를 발휘하려면 **배치(offline MAPF) 모드** 또는 **완전 수렴 MARL 정책**이 필요하다. 이는 온라인/오프라인 패러다임 불일치를 보여주는 중요한 연구 발견이다.

5.6 실험 II — LLM 를 자동 생성 결과

지표	결과
평가 입력 수	25개 (단순 6·복합 9·정렬 5·Fallback 5)
Zod 파싱 성공률	100% (25/25)
구조 불변성 통과율	100% (25/25)
평균 생성 시간	~2.1초
1회 재시도 발생	2/25 (8%) — 자동 복구
사용 모델	Claude Sonnet 4.5 (c l a u d e - s o n n e t - 4 - 5)

사전 Vector Store 없이 런타임 RuleDef 컨텍스트를 시스템 프롬프트에 동적 주입하는 **경량 컨텍스트 주입** 방식이 25입력 100% 통과율을 달성하였다.

6. AI 코딩 툴 활용 회고

6.1 Claude Code 활용 패턴

본 프로젝트는 2026-04-26부터 2026-05-31까지 약 5주간, **7개 세션·375개 프롬프트**를 통해 논문 실험 파트 전체를 Claude Code와 함께 구현하였다.

효과적이었던 패턴

패턴	설명	예시
계획 → 구현 분리	/plan 모드로 설계 확인 후 구현 시작	"CACTUS부터 하자 계획 세워줘" → Plan 승인 후 구현
단계적 알고리즘 추가	Strategy Registry 패턴 덕분에 기존 코드 수정 없이 신규 알고리즘 추가 가능	CBS-TS, CACTUS를 독립 모듈로 추가
학습 로그 공유	학습 중 에피소드 로그를 프롬프트로 공유해 수렴 상태 실시간 분석	$ep=19600 \quad \mu=-492 \quad \sigma=487$ → 학습 전략 수정
실측 수치 기반 논문 작성	실험 결과 CSV를 직접 분석해 논문 표·분석 작성	summary.csv → LaTeX 표 자동 생성
에이전트 분업	caie-paper-writer, caie-reviewer 등 특화 에이전트로 역할 분리	논문 심사위원 에이전트가 CAIE 기준 피드백 제공

한계 및 대응 방법

한계	대응
컨텍스트 창 초과 시 세션 재시작 필요	"This session is being continued..." 컨텍스트 요약으로 연속성 유지
학습 시간이 긴 RL 모델 수렴 미보장	스모크 테스트 + A* fallback으로 서비스 가용성 유지
대용량 출력 렌더링 이슈	파일 경유 저장·확인으로 우회
MILP 솔버 시간 초과	30초 제한 + greedy fallback

6.2 프롬프트 작성 팁

- 구체적인 목표 명시**: "SCI 저널 수준으로" → "CAIE 저널 수준으로 30 시드 × 3 시나리오 실험"
- 결과물을 함께 지정**: "실험 돌려줘" → "caie_experiment.py 출력을 summary.csv + statistical_tests.json으로"
- 오류·로그를 그대로 공유**: 학습 로그, 에러 메시지를 프롬프트에 직접 붙여넣기
- 점진적 검증**: 전체 실행 전 단위 테스트 → 소규모 시드 → 전체 30 시드 순서
- 도메인 용어 사용**: "충돌" 대신 "vertex/edge conflict", "교착" 대신 "deadlock timeout"

6.3 Claude Code가 특히 효과적이었던 영역

- 복잡한 수학적 알고리즘 구현**: QMIX Hypernetwork Mixer 단조성, Reverse Curriculum 진급 조건, MILP 수식 → Python 코드 변환
- 통계 검정 파이프라인**: Kruskal-Wallis + Wilcoxon + Holm-Bonferroni + rank-biserial r을 외부 라이브러리 없이 직접 구현(재현성 보장)
- Elsevier 논문 품질 그림·표**: 300 DPI, 서체·색상·레이아웃까지 CAIE 투고 기준에 맞게 자동화
- 코드 아키텍처 설계**: Strategy Registry, MetricsCollector, ExperimentRunner 등 확장 가능한 패턴 설계

7. 부록 — 산출물 위치

코드 산출물

분류	파일 경로	설명
AI 엔진	apps/ai-engine/main.py	FastAPI 서버
알고리즘	apps/ai-engine/engine/strategy.py	Strategy Registry
A*	apps/ai-engine/engine/astar.py	베이스라인
PPO	apps/ai-engine/engine/ppo_agent.py, route_env.py	RL 단일 에이전트
CACTUS	apps/ai-engine/engine/cactus/	QMIX MARL
CBS-TS	apps/ai-engine/engine/cbs_ts/	MILP+CBS+MLA*
시뮬레이터	apps/ai-engine/engine/simulation.py	SimPy + 12지표
실험	apps/ai-engine/scripts/caie_experiment.py	메인 실험
그림	apps/ai-engine/scripts/caie_figures.py	Figure 자동화
표	apps/ai-engine/scripts/caie_tables.py	LaTeX 표 자동화
통계	apps/ai-engine/engine/stats.py	Wilcoxon 등
실험 러너	apps/ai-engine/engine/experiment_runner.py	다중 시드 반복
테스트	apps/ai-engine/tests/	pytest 9종

실험 결과 산출물

apps/ai-engine/output/caie_20260522_221545/	← 최종 결과
├ raw_results.json	(30시드 × 3시나리오 × 4알고리즘 원시 수치)
├ summary.csv	(논문 표 작성용 CSV)
├ summary_stats.json	(Mean ± SD, CI)
├ statistical_tests.json	(KW + Wilcoxon + Holm-Bonferroni)
├ figures/	(fig1~10, PDF+PNG, 300 DPI)
└ tables/	(table1~6 + all_tables.tex)

문서 산출물

문서	경로
논문 초안	docs/paper-draft-CAIE.md (693줄)
논문 표 (LaTeX)	docs/paper-tables-latex.tex
논문 표 (HTML)	docs/paper-tables-hwp.html
알고리즘 방법론	docs/algorithms-methodology.md
CACTUS 방법론	docs/cactus-methodology.md

아키텍처 문서	docs/ARCHITECTURE.md
프롬프트 로그	PROMPTS.md (루트)
보고서 (본 문서)	docs/REPORT_AI_Coding_Tools.md

PROMPTS.md

전체 375개 프롬프트(실험 관련 119개 하이라이트 포함)는 **PROMPTS.md** (루트)를 참조.

본 보고서는 Claude Code(claude-sonnet-4-6)와의 대화를 통해 작성·검토되었습니다.